Does It Actually Run? An Execution-Based Reproducibility Study of Published Machine-Learning Finance Research

Akram Khan*

July 2026

Abstract

Reproducibility in empirical finance is normally measured by what a paper *says*: whether it claims code is available, and whether a link appears. That is a property of the manuscript, not of the research. This paper measures the property that matters to anyone who wants to build on published work, by taking the released artefacts and attempting to run them.

We introduce the Reproducibility Attrition Ladder, an ordered scale of eleven cumulative rungs running from "the paper makes no code-availability claim" to "an emitted number matches the paper's headline result". Every rung except the last is decided mechanically, and every failure is recorded against a closed vocabulary of reasons, so the point at which an artefact stops, and the reason it stops, are both reportable as data rather than as anecdote. We apply the ladder to a corpus of arXiv quantitative-finance machine-learning papers, cloning each advertised repository, building its declared environment inside a container, and executing its entry point under a fixed time budget.

Applied to 459 arXiv quantitative-finance machine-learning papers that advertise a code repository, the curve falls steeply and not where the reproducibility discourse usually points. Link rot removes 5.2%. The two large losses are environment declaration, where 176 of 427 cloned repositories ship no dependency specification at all, and environment construction, where half of those that do declare dependencies can no longer install them, through packages that have left the index, pin sets that no longer resolve, and sources that no longer compile. Thirteen

*Independent Researcher. ORCID: 0009-0002-7521-8648. Email: 1819ak@gmail.com. The measurement harness, the corpus manifest, and the full per-paper outcome records are released at github.com/ayk5511/exec-repro.

point nine per cent yield an identifiable entry point and **seven papers, 1.5%, execute to completion and emit output** without correspondence, manual repair, or reading beyond the README. The most discretionary step, deciding what to run, is reported under two specifications that agree exactly on every rung decided before it applies, and the mechanically determined rungs replicate on an earlier, independently drawn sample.

The contrast with the existing finance result is the point. Running the code of 168 coordinated research teams on supplied raw data reproduces the published answer 52% of the time (Pérignon et al., 2024); starting instead from the URL a published paper gives its readers, with no data supplied and no author contact, 2% of artefacts run, and 1.5% emit output. Most of what computational reproducibility measures in finance depends on conditions an ordinary reader does not have.

The finding is about artefacts rather than findings: released code that does not run unassisted is not thereby wrong, but it cannot be checked without reimplementation. Because the dominant environment failure is a dependency leaving the index rather than an omission at submission, executability decays after publication, and the same corpus will score lower each year.

The measurement instrument is itself released as an installable package, so the study can be repeated on other corpora, re-run as the literature changes, and extended to fields beyond finance. We are explicit about what the ladder does and does not establish: an artefact that fails to run is not thereby wrong, it is not independently executable as released, which is a weaker but more precisely measurable claim.

Keywords: reproducibility, computational reproducibility, research artefacts, machine learning, quantitative finance, meta-science, open science, research software

JEL Classification: C63, C88, G17

Contents

1 Introduction **4**

1.1 The instrument 4

1.2 Measuring the measurement 5

1.3 Contributions 5

1.4 Roadmap 6

2 Related work **6**

2.1 Three literatures, three questions 6

2.2 Execution-based studies 7

2.3 What is new here 7

3 The Reproducibility Attrition Ladder **8**

3.1 Why a ladder rather than a score 8

3.2 The rungs 8

3.3 A closed vocabulary for failures 9

3.4 Execution environment 10

3.5 What the ladder does not measure 11

4 Results **11**

4.1 The corpus 11

4.2 The attrition curve 12

4.3 Why artefacts stop 13

4.4 Sensitivity to the entry-point heuristic 14

4.5 Replication on an independent sample 14

4.6 Rung 10 14

5 Discussion **15**

5.1 Reading this against the coordinated-study result 15

5.2 What the curve licenses us to say 15

5.3 Where the attrition actually happens 16

5.4 Limitations 16

5.5 The harness as the durable contribution 17

5.6 Future work 17

1 Introduction

Machine learning is now a core empirical methodology in quantitative finance (Gu et al., 2020), which makes its results unusually dependent on code: the method is the implementation, and a description in prose rarely determines it. A reader who wants to build on such a result therefore has a concrete task: obtain the authors' code, install what it needs, run it, and see whether the numbers come out. Almost everything the reproducibility literature measures is upstream of that task. Instruments count whether a paper states that code is available, whether a link is present, whether data are described, whether an environment is specified. These are properties of the manuscript. They are cheap to score at scale, they are useful, and they are not the same thing as being able to run the code.

The gap matters because the two can diverge in both directions. A paper can disclose exemplary metadata, a repository link, a licence, a `requirements.txt`, and still be unusable, because the pinned dependencies no longer resolve, because the entry point expects a data file that was never shipped, or because the repository was deleted after publication. Conversely a sparse repository with no formal environment specification may run without incident.

Finance does have a direct measurement of computational reproducibility. Pérignon et al. (2024) ran the code of 168 research teams on the same raw data and found that it regenerates exactly the same results 52% of the time. That figure is a best case by construction: the code was written for a coordinated study, the data were supplied, and the teams were reachable. A reader of a published paper has none of those advantages. They have a URL.

This paper measures the cold-start case. We take a corpus of machine-learning papers in quantitative finance, extract every code URL the authors advertise, and attempt to run what we find: resolve the link, clone the repository, build the declared environment inside a container, locate an entry point, and execute it under a fixed time budget. What we record for each artefact is the furthest point it reached and the reason it stopped.

1.1 The instrument

Our first contribution is the measurement design itself. Reproducibility is usually reported as a score, which conflates failures with nothing in common: a deleted repository and an unresolvable dependency both cost a point, and the aggregate cannot distinguish a hosting problem from a packaging problem. We instead use an ordered, cumulative ladder of eleven rungs (Section 3), running from "the paper makes no code-availability claim" to "an emitted number matches the paper's headline result". Each artefact is pushed as far as it will go; the distribution of stopping points is the attrition curve, and the reasons for stopping form a second, orthogonal table.

Two properties of the design carry most of its weight. Because the rungs are cumulative, the curve is monotone by construction, and a violation of monotonicity is a detectable scoring inconsistency rather than a quirk of the data; the audit script checks it on every build. Because failure reasons are drawn from a closed vocabulary rather than free text, the failure mix at each rung is itself reportable, so we can say not only how many artefacts stopped at environment construction but what fraction of those were caused by a package disappearing from the index rather than by a version pin that has aged out.

1.2 Measuring the measurement

A study of this kind carries an unusual risk: a defect in the harness produces a number that looks like a finding. The failure modes are not hypothetical. During development, a container backend that started a fresh container for the install step and another for the execution step produced a clean, plausible result in which no artefact in the corpus executed at all; every dependency installed in the first container had been discarded before the second one ran. The number was publishable-looking and entirely an artefact of our own code. It was caught because the failures took a tenth of a second, which is not how real machine-learning code fails.

We therefore treat the harness as an object of measurement too. Failures attributable to our tooling carry their own reason code and are excluded from every claim about the literature. Conditions that indicate the harness is misconfigured, rather than that a paper is deficient, raise an error instead of being recorded as data. The entry-point heuristic, the most discretionary part of the pipeline, is run in two configurations and both curves are reported, so a reader can see how much of the result is the literature and how much is our own conservatism. Section 5 returns to what this does and does not license us to conclude.

1.3 Contributions

- The Reproducibility Attrition Ladder: an ordered, cumulative scale for execution-based reproducibility, with a closed failure vocabulary that makes the reasons for attrition reportable rather than anecdotal.
- An execution study of a corpus of arXiv quantitative-finance machine-learning papers, reporting the attrition curve, the failure mix at each rung, and a sensitivity analysis over the one heuristic that materially affects the upper rungs.
- An open, installable harness that performs the measurement, so the study can be repeated, extended to other corpora, and re-run as the literature changes. The per-paper outcome records are released in full.

1.4 Roadmap

Section 2 places the work against the reproducibility literature in science generally, in empirical finance, and in machine learning. Section 3 defines the ladder and the execution environment. Section 4 reports the corpus, the attrition curve, and the failure mix. Section 5 discusses what the curve implies for readers, authors, and editors, and is explicit about the limits of the claim.

2 Related work

2.1 Three literatures, three questions

The word "reproducibility" names different problems in different fields, and the distinction determines what our measurement can and cannot speak to.

The broadest strand asks whether published findings are *true*. Ioannidis (2005) argued on statistical grounds that most published research findings are false, and the Open Science Collaboration (2015) supplied direct evidence by rerunning a hundred psychology experiments with fresh data. In empirical finance the corresponding work is Harvey et al. (2016), who recalibrate significance thresholds for a literature that has tested hundreds of candidate return predictors, and Hou et al. (2020), who rebuild the anomaly literature under uniform methodology and find much of it does not survive. These studies concern the durability of findings under new data or stricter statistics. They are not about whether the original code runs.

A second strand asks whether an artefact is *documented* well enough that reproduction is possible in principle. Gundersen and Kjensmo (2018) score artificial-intelligence papers on documentation dimensions and find most underspecified; the Pineau et al. (2021) NeurIPS programme turned similar concerns into submission-time policy, introducing checklists and artefact-availability expectations at a major venue. Instruments in this family, including the disclosure rubric applied to machine-learning finance by Khan (2026b) and the audit-trail dimensions of Khan (2026a), are scored by reading the paper and inspecting the repository. They are cheap enough to apply at corpus scale, and they measure disclosure.

The third question, and the one this paper takes up, is whether the released artefact *executes*. It sits between the other two: passing it is neither necessary nor sufficient for a finding to be true, but failing it means no reader can check the finding without reimplementing the work. The engineering literature explains why this is hard even for competent authors. Sculley et al. (2015) document how machine-learning systems accumulate dependencies, configuration, and undeclared coupling faster than conventional software, so a repository that ran on its author's machine drifts out of executability without anyone touching it.

2.2 Execution-based studies

Two lines of work already run published code, and the present study is defined by how it differs from each.

The first is at scale but outside finance. Trisovic et al. (2022) took more than 2,000 replication datasets from the Harvard Dataverse, comprising over 9,000 R files, and executed them in a clean environment: 74% of files failed on first execution, falling to 56% after automated code cleaning. That is the closest methodological relative to what we do, and it establishes that large-scale unassisted execution is both feasible and informative. It is confined to R and to deposited replication packages rather than to repositories advertised in papers, and it reports a single failure rate rather than locating where in the pipeline the failure occurred.

The second is in finance but under very different conditions. Pérignon et al. (2024) ran the code of 168 research teams answering six prespecified questions, and found that running that code on the same raw data regenerates exactly the same results only 52% of the time. This is the definitive finance result on computational reproducibility, and it is a *best-case* measurement by construction: the code was written by participating teams for the purpose, the raw data were supplied, and the study was coordinated. Those are precisely the conditions an ordinary reader of a published paper does not have.

Computer science has also institutionalised artefact evaluation at systems and security venues, where committees run submitted artefacts and report substantial attrition between what is advertised and what builds. That work is a human, pass/fail judgement made by a reviewer who may correspond with the authors, which makes it stronger evidence about an individual artefact and weaker evidence about a distribution.

2.3 What is new here

The gap this paper fills is the unassisted, cold-start case in finance: not code supplied for a coordinated study with data attached, but whatever a published paper points a reader to, retrieved from that pointer, on a clean machine, with no data supplied and no author contact. Read against Pérignon et al. (2024), the question is not whether finance code reproduces under favourable conditions, but how much of that 52% survives the removal of the favourable conditions.

Three things are new. The ladder supplies an ordered scale where the literature has used pass/fail or a single rate, so attrition becomes a curve with a diagnosable shape rather than one number: our result is not simply that little runs, but that most of the loss occurs at environment declaration and construction rather than at link rot or at execution itself. The closed failure vocabulary makes the *reasons* a reportable table rather than a narrative. And the harness is released as an instrument, which matters because the quantity decays: a corpus scored today will score lower in two years as packages are

yanked and repositories are archived, and a study that cannot be re-run cannot track that.

3 The Reproducibility Attrition Ladder

3.1 Why a ladder rather than a score

Reproducibility instruments in common use return a score: a paper earns points for releasing code, for releasing data, for pinning an environment. Scores are convenient, but they conflate failures that have nothing to do with one another. A paper whose repository has been deleted and a paper whose repository is complete but whose dependencies no longer resolve both lose the same point, and the aggregate number cannot tell the two apart. The remedies differ completely: one is a hosting problem, the other is a packaging problem.

An ordered ladder keeps that information. Each artefact is pushed as far as it will go, and what we record is the rung at which it stopped together with the reason it stopped. Because the rungs are cumulative, reaching rung n implies every rung below n was cleared, so the statement "x% of papers clear rung n or better" is unambiguous and the resulting curve is monotone by construction. Monotonicity is not decoration: it is a property the audit script checks on every build, because a non-monotone curve is a signal that the corpus was scored inconsistently.

3.2 The rungs

Table 1 gives the scale. Rungs 0 to 2 are properties of the manuscript and require no execution. Rungs 3 and 4 test retrieval. Rungs 5 and 6 test whether the declared environment can be built. Rungs 7 to 9 test execution. Rung 10 is the scientific claim.

Table 1: The Reproducibility Attrition Ladder. Rungs are cumulative and strictly ordered. Rungs 0 to 9 are decided mechanically; rung 10 requires reading the paper for a headline number and is scored only on a manually curated subset.

#	Rung	Cleared when
0	`NO_CLAIM`	(floor) the paper makes no code-availability claim
1	`CLAIM_ONLY`	the paper claims code exists but gives no pointer
2	`URL_PRESENT`	a code URL appears in the paper or its metadata
3	`URL_RESOLVES`	the URL still answers: no link rot
4	`CLONED`	the repository can be retrieved
5	`DEPS_DECLARED`	a dependency specification exists
6	`ENV_BUILDS`	those dependencies actually resolve and install
7	`ENTRYPOINT_FOUND`	a runnable entry point is identifiable
8	`RUNS`	the entry point completes with exit code 0 in budget
9	`PRODUCES_OUTPUT`	execution emits artefacts or numeric output
10	`MATCHES_PUBLISHED`	an emitted number matches a headline number

Two design choices deserve defending.

First, *rung 10 is never asserted by the harness.* Deciding whether an emitted number corresponds to a paper's headline result requires reading the paper, locating the claim, and judging whether the quantities are comparable at all. That is human work. The harness therefore stops at rung 9, the audit script verifies that no record claims rung 10 automatically, and the rung-10 figure is reported only for a manually curated subset and never extrapolated to the corpus.

Second, *network access is denied during execution and permitted during installation.* A repository that downloads its data when it runs is not self-contained: a reader without the author's credentials, or reading after the endpoint moves, cannot reproduce the result. Allowing the fetch would let our own connectivity paper over that. Installation is a different matter, since package indexes are the normal mechanism by which environments are built, so the network is available there.

3.3 A closed vocabulary for failures

Recording *that* an artefact failed at rung 6 is less useful than recording *why.* Free-text error capture makes that unaggregatable, so each rung carries a fixed set of admissible reasons and the data structure refuses any value outside it. The result is that the failure mix at each rung is a table.

The vocabulary separates causes that call for different responses. At rung 6, for instance, `package_not_found` (a dependency has left the index), `python_version_unsupported`

(the pin has aged out of the supported range), and `build_from_source_failed` (a wheel is unavailable for the platform) are distinct, and only the first is really about the paper's own choices.

One category is reserved for our own defects. If the harness locates a dependency specification and then fails to read it, that is a bug in the measurement, not a property of the paper, and it is recorded as `spec_unreadable_by_backend` and excluded from every claim about the literature. This is not hypothetical: an early version of the container backend passed a bare filename for specifications found in a subdirectory, and the resulting installer error was initially classified as a dependency conflict, that is, as evidence against the papers. The separate code, and a regression test pinning it, exist because that failure mode is easy to reintroduce and impossible to notice in an aggregate.

3.4 Execution environment

Each repository is cloned to a scratch directory, its declared environment is built, and its entry point is executed inside a container with a fixed CPU and memory allocation. Containers matter for two reasons: dependency isolation, so that one repository's pins cannot affect another's, and process isolation, since we are running code written by authors who never anticipated a stranger executing it. Where a container runtime is unavailable the harness falls back to a per-repository virtual environment combined with kernel-level sandboxing, which confines filesystem writes and denies network but shares the host kernel. The backend used is recorded on every result, and results produced under different isolation regimes are reported separately rather than pooled.

Entry-point selection is the most discretionary step in the pipeline, so it is run as a switch rather than a constant. The *convention* route looks for conventionally named scripts and otherwise accepts a single unambiguous top-level script; build scripts and configuration modules are excluded, because executing `setup.py` exercises the packaging toolchain rather than the paper's analysis. The *documentation* route additionally reads the repository's README and takes the command the authors themselves give, subject to three restrictions: the command must name a target that exists in the repository, it must not be an installation or testing step, and it must not be a compound shell pipeline, since those cannot be reproducibly parsed.

Both routes exclude directories whose contents are never the analysis: test suites, example folders, documentation, and vendored dependencies. That exclusion is not hypothetical tidiness. Two repositories in this corpus ship a copy of a Chinese-tokenisation library, and its bundled `test/demo.py` satisfied the naming heuristic; the harness executed the library's demo and recorded its output as though the paper's pipeline had produced it. Both were counted at the top of the ladder until the outputs were read and found to be identical to each other and unrelated to either paper. Running a repository's

tests is also not running its analysis, which is why a documented `pytest` invocation is refused as an entry point.

Both configurations are run over the whole corpus and both curves are reported (Section 4.4). Neither matches a determined human who would read the source and infer the intended invocation, so both are lower bounds; reporting the pair makes the size of that discretion visible instead of burying it in a design choice. Where no entry point is identified under the active configuration, the harness records `no_entrypoint_identifiable` rather than guessing: a harness that invents plausible commands would report their failure as the paper's.

3.5 What the ladder does not measure

An artefact that stops low on the ladder is not thereby wrong. It is not independently executable as released, on a current platform, within our time budget, by a reader who did not write it. That is a narrower claim than "irreproducible" and it is the only claim the instrument supports. Conversely, clearing rung 9 shows that code ran and produced output; it says nothing about whether the output is correct, whether the method is sound, or whether the finding replicates on new data. Rung 10 speaks to the first of those and to nothing beyond it.

4 Results

4.1 The corpus

The corpus is 1,754 machine-learning papers in quantitative finance, harvested from arXiv along two axes. The six quantitative-finance categories (q-fin.CP, ST, TR, PM, RM, MF) were read to exhaustion under a machine-learning filter, and the general machine-learning categories (cs.LG, stat.ML, econ.EM, cs.CE) were queried by finance topic, which is where finance work that is not indexed under q-fin lives. Of these papers, 459 advertise a code URL on a recognised hosting service and constitute the execution sample.

Code URLs were recovered in two passes, and the difference between them is itself a result. Reading abstracts alone, the standard method, found 98. Reading the rendered full text of each paper found a further 361, so nearly four-fifths of the advertised repositories in this literature are invisible to abstract-level extraction. Any disclosure rate computed from abstracts is a severe underestimate.

Extracted URLs required normalisation before use. URLs harvested from prose routinely carry a trailing sentence period or a wrapping bracket, and treating those characters as part of the address would have reported live repositories as dead.

Two features of the corpus bound the claims. It is an arXiv sample, so venues whose

authors do not preprint are absent, and it is described throughout as arXiv ML-finance rather than as the ML-finance literature. And although the harvest reaches back to 2019, it remains weighted toward recent work.

4.2 The attrition curve

Table 2 reports the fraction of the 459-paper execution sample clearing each rung. The curve is monotone by construction and is verified as such by the audit script.

Table 2: Attrition across the ladder, $n = 459$ papers advertising a code URL. The *README* column is the primary specification, in which entry points may be taken from the authors' own documentation; the *convention* column restricts entry points to conventionally named scripts. The two agree exactly through rung 6, which are decided before any entry-point heuristic applies.

#	Rung	README (primary)		Convention only	
		n	%	n	%
2	`URL_PRESENT`	459	100.0	459	100.0
3	`URL_RESOLVES`	435	94.8	435	94.8
4	`CLONED`	427	93.0	427	93.0
5	`DEPS_DECLARED`	251	54.7	251	54.7
6	`ENV_BUILDS`	127	27.7	127	27.7
7	`ENTRYPOINT_FOUND`	64	13.9	39	8.5
8	`RUNS`	9	2.0	4	0.9
9	`PRODUCES_OUTPUT`	7	1.5	2	0.4

Three features of the curve carry the paper's content.

Link rot is real but modest. Twenty-four of 459 advertised repositories (5.2%) no longer resolve. This is the failure mode most often assumed to dominate, and it is the smallest of the large drops. Because a transient resolver failure is indistinguishable from a dead domain on a single attempt, transport-level failures are retried before being recorded; without that, one repository in this corpus would have been reported as rotted when it was live.

The first large drop is environment declaration. Of the 427 repositories that clone, 176 ship no dependency specification of any kind, taking the sample from 93.0% to 54.7% in a single rung. Nothing exotic is required to clear it: a `requirements.txt` suffices.

The second large drop is environment construction. Of the 251 repositories that declare dependencies, half again fail to install them, taking the sample to 27.7%. This is substantially decay rather than negligence, and Table 3 shows the mechanism.

4.3 Why artefacts stop

Table 3: Failure mix by blocking rung, primary specification. Counts are repositories stopping at that rung for that reason.

Rung	Reason	n
URL_RESOLVES	http_404	23
	http_403_private	1
CLONED	not_a_repository	5
	clone_timeout	2
	clone_auth_required	1
DEPS_DECLARED	no_dependency_spec	176
ENV_BUILDS	package_not_found	45
	dependency_resolution_conflict	38
	build_from_source_failed	34
	non_python_toolchain_required	5
	python_version_unsupported	2
ENTRYPOINT_FOUND	no_entrypoint_identifiable	63
RUNS	runtime_exception	32
	missing_data_file	16
	execution_timeout	4
	requires_gpu	2
	requires_api_key	1
PRODUCES_OUTPUT	ran_but_no_artefact	2

The environment failures divide into three roughly comparable mechanisms. `package_not_found` (45) is a pinned dependency that has left the index; `dependency_resolution_conflict` (38) is a pin set that no longer admits a solution; `build_from_source_failed` (34) is a package with no current wheel that fails to compile. All three are properties of elapsed time rather than of the authors' diligence at submission, which means executability decays after publication. It also means this measurement is timestamped: the same corpus scored two years hence will score lower.

At the execution rung, the sixteen `missing_data_file` outcomes are substantive. Network access is denied during execution by design, so a repository that fetches its inputs at run time is recorded here rather than being rescued by our connectivity: the artefact as released does not carry what it needs.

4.4 Sensitivity to the entry-point heuristic

Identifying what to run is the most discretionary step, so the study is run in two configurations. Restricting to conventionally named scripts finds an entry point in 39 repositories; additionally accepting a command documented in the repository's own README finds 64. The looser specification raises the number that execute from 4 to 9 and the number that emit output from 2 to 7.

The two configurations agree exactly on rungs 2 through 6, which are decided before any entry-point logic runs. That agreement is checked by the audit script and is the study's internal-consistency control: a discrepancy there would mean the campaign was not deterministic and no figure above could be quoted.

The gap between the columns is the honest measure of how much of the upper curve is our conservatism rather than the literature's. It is large in relative terms and small in absolute terms: on either specification, fewer than one paper in fifty executes to completion unassisted.

4.5 Replication on an independent sample

An earlier campaign scored a 154-paper sample drawn only from the q-fin categories. The present corpus is three times larger and reaches papers through the general machine-learning categories, which the earlier sample could not see.

The comparison is restricted to rungs 2 through 6, which are decided before any entry-point logic applies and are therefore unaffected by the heuristic revisions made between the two campaigns. On those rungs the samples agree closely: resolution 92.2% against 94.8%, cloning 87.7% against 93.0%, dependency declaration 53.2% against 54.7%, and environment construction 26.6% against 27.7%. Both of the paper's principal findings, the size of the declaration drop and the size of the construction drop, sit inside this range.

We do not claim replication above rung 6. The two campaigns used different entry-point rules, the later one stricter, so the upper figures are not like-for-like and the difference between them measures our revision rather than the literature.

Because the second sample is larger and differently sourced, the agreement on the mechanically determined rungs is evidence that the shape of the curve is a property of the literature rather than of the particular papers first drawn.

4.6 Rung 10

Seven repositories produced output and are therefore eligible for the final rung, which asks whether an emitted number matches a headline number in the paper. Seven is too few to support a rate, and we report no percentage for rung 10. The worksheet pairing each artefact's captured stdout with its paper's numeric claims is released with the corpus

so the adjudication can be inspected and disputed.

That the sample thins to seven from 459 is itself the finding: the question the rung was designed to answer is, for this corpus, almost entirely unanswerable, because almost nothing gets far enough to ask it of.

5 Discussion

5.1 Reading this against the coordinated-study result

The natural comparison is Pérignon et al. (2024), who report 52% exact reproduction in finance. Nothing here contradicts that number; the two measure different situations, and the distance between them is the useful quantity.

In the coordinated setting, code is written by a participating team knowing it will be run, the raw data are supplied, and the researchers are reachable. In ours, the artefact is whatever a published paper points its readers to, retrieved cold, on a clean machine, with no data supplied and nobody to ask. The first measures whether finance code *can* reproduce. The second measures whether a reader can get it to run at all.

That the figures are 52% and 2% does not mean the coordinated result was optimistic. It means most of what that measurement captures is supplied by conditions surrounding the code rather than by the artefact itself: the data, the authors' attention, the knowledge that someone will check. Strip those away and what remains is the artefact as released, which is what a reader inherits. The practical implication in Section 5.3 follows directly: the interventions that would move our number are the ones that make an artefact self-contained, not the ones that make an author more careful at submission.

5.2 What the curve licenses us to say

The measured quantity is narrow and worth stating precisely. Of 459 quantitative-finance machine-learning papers that advertise a code repository, seven produced output when a stranger cloned the repository, built the declared environment in a container, and ran the documented entry point without correspondence, manual repair, or reading beyond the README.

That is not a claim that the other 452 papers are wrong, and it is not a replication failure rate. It is a statement about artefacts, not findings: the released code, as released, does not run unassisted on a current platform. Several of the failures are ones an author could resolve in a message and a motivated reader in an afternoon. The claim is about the cost of that afternoon, multiplied across a literature.

The converse also holds. Producing output says nothing about whether the output is correct, whether the method is sound, or whether the result would survive new data.

Rung 10 addresses the first of those and, in this corpus, has almost no sample to work with.

5.3 Where the attrition actually happens

The practical value of an ordered ladder is that it localises the loss, and the location is not where the reproducibility discourse usually points. Link rot, the most-discussed failure, accounts for 5.2%. The two large drops are environment declaration (176 repositories ship no dependency specification) and environment construction (half of those that declare one no longer install).

These have different remedies. The first is a submission-time omission and is cheap to fix: a `requirements.txt` clears it, and an editorial checkbox would move the number. The second is decay, and no amount of author diligence at submission prevents it, because the failure is that a pinned dependency later left the index. Mitigating it requires artefacts that pin more than package versions, which in practice means a container image or a lockfile with hashes.

The distinction matters for policy. A journal that requires a dependency file addresses the larger drop immediately. A journal that wants artefacts to survive five years has to ask for something stronger, and should know it is asking for something more expensive.

5.4 Limitations

The entry-point heuristic is the binding constraint on the upper rungs. Between rungs 6 and 8 the measurement is partly measuring us. We report both specifications (Section 4.4) rather than choosing one, but neither matches a determined human who would read the source, infer the intended invocation, and supply arguments. The upper rungs are therefore a lower bound on what a persistent reader could achieve.

The corpus is arXiv and recent. Fields whose authors do not preprint are absent, and the sample is weighted toward the last two years. The curve describes current arXiv practice in quantitative finance.

The measurement is timestamped. Because `package_not_found` dominates the environment failures, the same corpus will score lower each year without a single repository changing. Any comparison against these figures must be dated, which is part of why the harness is released rather than only the numbers.

Time and resource budgets bind. Execution is capped at fifteen minutes per artefact on two allocated cores without a GPU. Four repositories hit that ceiling and are recorded as timeouts, not failures of a different kind. An artefact whose honest runtime is a day on a cluster is not distinguishable here from one that hangs.

Rung 10 is human-scored. The final rung reflects one scorer's judgement about whether an emitted number corresponds to a published claim. With seven eligible arte-

facts this is disclosed as adjudication rather than measurement, and the worksheet is released so that the judgements can be checked.

5.5 The harness as the durable contribution

The numbers in Section 4 describe one corpus at one moment. The instrument that produced them is the part intended to outlast the snapshot: it is installable, it takes any corpus of paper identifiers and code URLs, and it emits per-artefact records with a closed failure vocabulary.

Three properties were built in because their absence would have been invisible in the output. Harness-attributable failures carry a distinct reason code and are excluded from claims about the literature. Conditions indicating harness misconfiguration raise an error rather than being recorded as data, so a broken run fails loudly instead of producing a clean-looking corpus of false negatives. And the discretionary heuristic is a switch rather than a constant, so its effect is reportable instead of assumed.

Those choices were not precautionary in the abstract. During development, a container backend that installed dependencies in one container and executed in another produced a complete, plausible, entirely false result in which nothing in the corpus ran; it was caught because the failures took a tenth of a second. A study that measures how often other people's code silently fails to work is poorly placed to assume its own does.

5.6 Future work

Three extensions are supported by the released artefact. The corpus can be widened beyond arXiv and beyond finance, since nothing in the ladder is domain-specific. The same corpus can be re-scored periodically to measure the decay rate directly, converting a limitation into a result. And the entry-point step can be strengthened, ideally by comparing the automated heuristics against a human-annotated subset, which would replace the sensitivity band with a calibration.

References

Gu, S., Kelly, B., and Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5):2223–2273.

Gundersen, O. E. and Kjensmo, S. (2018). State of the art: Reproducibility in artificial intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.

Harvey, C. R., Liu, Y., and Zhu, H. (2016). . . . and the cross-section of expected returns. *Review of Financial Studies*, 29(1):5–68.

Hou, K., Xue, C., and Zhang, L. (2020). Replicating anomalies. *Review of Financial Studies*, 33(5):2019–2133.

Ioannidis, J. P. A. (2005). Why most published research findings are false. *PLoS Medicine*, 2(8):e124.

Khan, A. (2026a). Auditing the audit trails: A cryptographic framework and a compliance survey of 185 published machine-learning papers in quantitative finance. Technical Report 6932243, SSRN.

Khan, A. (2026b). Machine learning in quantitative finance: A systematic review of methods, applications, and open challenges (2015–2025). Technical Report 6562398, SSRN.

Open Science Collaboration (2015). Estimating the reproducibility of psychological science. *Science*, 349(6251):aac4716.

Pérignon, C., Akmansoy, O., Hurlin, C., Dreber, A., Holzmeister, F., Huber, J., Johannesson, M., Kirchler, M., Menkveld, A. J., Razen, M., and Weitzel, U. (2024). Computational reproducibility in finance: Evidence from 1,000 tests. *The Review of Financial Studies*, 37(11):3558–3593.

Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché Buc, F., Fox, E., and Larochelle, H. (2021). Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 2503–2511.

Trisovic, A., Lau, M. K., Pasquier, T., and Crosas, M. (2022). A large-scale study on research code quality and execution. *Scientific Data*, 9:60.